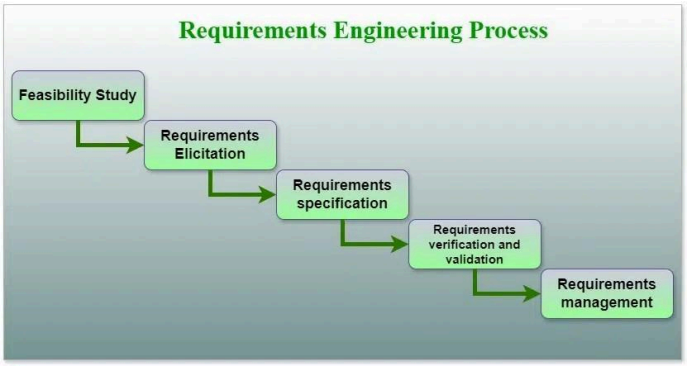
IMPORTANT QUESTIONS WITH ANSWERS
UNIT II

Syllabus: Requirement analysis and specification – Requirements gathering and analysis – Software Requirement Specification – Formal system specification – Finite State Machines – Petrinets – Object modelling using UML – Use case Model – Class diagrams – Interaction diagrams – Activity diagrams – State chart diagrams – Functional modelling – Data Flow Diagram- CASE TOOLS.

Part A

1. Write distinct steps in requirement engineering process. OR List out requirements engineering.

Ans: The requirement engineering process generally involves these distinct steps: Elicitation, Analysis, Specification, Validation, and Management. These steps ensure that the requirements are well-defined, understood, and maintained throughout the project lifecycle.



1. Elicitation: This is the initial phase where requirements are gathered from various stakeholders (users, clients, domain experts). Techniques include interviews, surveys, workshops, and document analysis.

2. Analysis: Once requirements are gathered, they are analyzed for feasibility, consistency, completeness, and clarity. This step identifies potential conflicts and ambiguities.

3. Specification: In this step, the requirements are documented in a clear and unambiguous manner, often using a requirements specification document. This document serves as a contract between stakeholders and the development team.

4. Validation: This step ensures that the documented requirements accurately reflect the needs of the stakeholders and are feasible to implement. Validation techniques include reviews, prototyping, and testing.

5. Management: This is an ongoing process throughout the project lifecycle. It involves tracking changes to requirements, maintaining traceability, and ensuring that all stakeholders are aware of the current requirements.

2. List the characteristics of good SRS. Why SRS Must be traceable? What is traceability?

Ans: A good Software Requirements Specification (SRS) document should be correct, complete, consistent, unambiguous, verifiable, modifiable, traceable, and ranked for importance and stability. These characteristics ensure the SRS accurately reflects the system's requirements and facilitates effective development and testing.

- **Correctness:**The SRS accurately reflects the intended functionality and behavior of the system.
- **Completeness:**The SRS includes all necessary information for development, leaving no room for assumptions or ambiguities.
- **Consistency:**There are no contradictions or conflicts within the SRS document.

- **Unambiguousness:**The requirements are clearly and precisely stated, leaving no room for misinterpretation.
- **Verifiability:**It should be possible to verify the requirements through testing or other means.
- **Modifiability:**The SRS can be easily updated to accommodate changes in requirements.
- **Traceability:**Requirements can be traced back to their origins and forward to their implementation.
- **Ranking for importance and stability:**Requirements are prioritized based on their importance and stability over time, which helps in managing changes and development efforts.

A Software Requirements Specification (SRS) must be traceable to ensure that all requirements are clearly defined, linked to their origins, and can be tracked throughout the development lifecycle. Traceability means being able to follow a requirement from its initial source to its implementation and verification, and back again. This ensures that no requirements are missed, that changes are properly managed, and that the final product meets all specified needs.

Traceability, in the context of software development, refers to the ability to track the relationships between different elements of the project. In an SRS, this means being able to follow the journey of each requirement from its initial definition to its implementation in the code and its eventual verification through testing.

Why Traceability is Important for an SRS:

1. **Verification and Validation:**Traceability allows for verification that each requirement has been addressed and validated through design, implementation, and testing.
2. **Change Management:**When changes are needed, traceability helps identify which parts of the system are affected by a change, enabling impact analysis and efficient change management.
3. **Risk Management:**By tracing risks back to their originating requirements, traceability helps in identifying and mitigating potential problems early in the development process.
4. **Improved Quality:**Ensuring that all requirements are traceable leads to a more robust and reliable product by minimizing the risk of overlooking critical aspects.
5. **Compliance:**In regulated industries, traceability is often a mandatory requirement to demonstrate compliance with standards and regulations.
6. **Better Communication:**Traceability provides a clear understanding of the relationships between requirements and other project artifacts, facilitating better communication and collaboration among stakeholders.
7. **Auditability:**Traceability creates an audit trail, making it easier to track the history of requirements and demonstrate compliance during audits.

3. Differentiate data flow diagram and state transition diagram.

Ans: State Transition diagrams are used to model the behavior of one object over several use cases in the system. A state describes the condition of an object at a given point in its lifetime. A state diagram describes all possible states of an object, and the paths to get from one state to another when an external event is received. But, a Data Flow Diagram (DFD) is a graphical representation of flow of information or data throughout the system

Feature	Data Flow Diagram (DFD)	State Transition Diagram (STD)
Focus	Flow of data	States and transitions
Purpose	Visualize data flow	Model system behavior

Elements	Processes, data stores, external entities, data flows	States, transitions, events
Example	Online order processing	Vending machine
Type of Modeling	Data-oriented	Behavior-oriented

4. Define functional and non- functional requirements.

Ans: Functional Requirements: Describe the specific behaviors and actions the system must perform. They are often expressed as user stories, use cases, or system features.

Examples:

- "The system must allow users to log in with a username and password."
- "The system must enable users to search for products by name."
- "The system must generate a report of all transactions for a specific date range."

Key aspect: Focus on what the system does.

Non-Functional Requirements**:** Specify the quality characteristics of the system. They define how well the system performs its functions.

Examples:

Performance: "The system must respond to user requests within 2 seconds."
Security: "The system must encrypt all sensitive data."
Usability: "The system must be easy to navigate for new users."
Scalability: "The system must be able to handle 10,000 concurrent users."
Reliability: "The system must have a 99.99% uptime."

NON-FUNCTIONAL VS FUNCTIONAL REQUIREMENTS

	Non-functional Requirements	Functional Requirements
Objective	How the product works	What the product does
Focus	Focus on user expectations	Focus on user requirements
Documentation	As a quality attribute	In use case
End Result	Product properties	Product features
Essentiality	Not mandatory, but desirable	Mandatory
Testing	Performance usability, security testing, etc. Tested after functional testing	Component, API, UI testing, etc. Tested before non-functional testing
Types	Operational, revisional, and transitional	External interface, authentication, authorization levels, and business rules

5. What is the need for feasibility analysis?

Ans: A feasibility study, also known as a feasibility analysis or report, is an assessment conducted to determine the likelihood of a project's success.

A feasibility study is a comprehensive evaluation of a proposed project that evaluates all factors critical to its success to assess its likelihood of success. Business success can be defined primarily in terms of ROI, the amount of profits the project will generate.

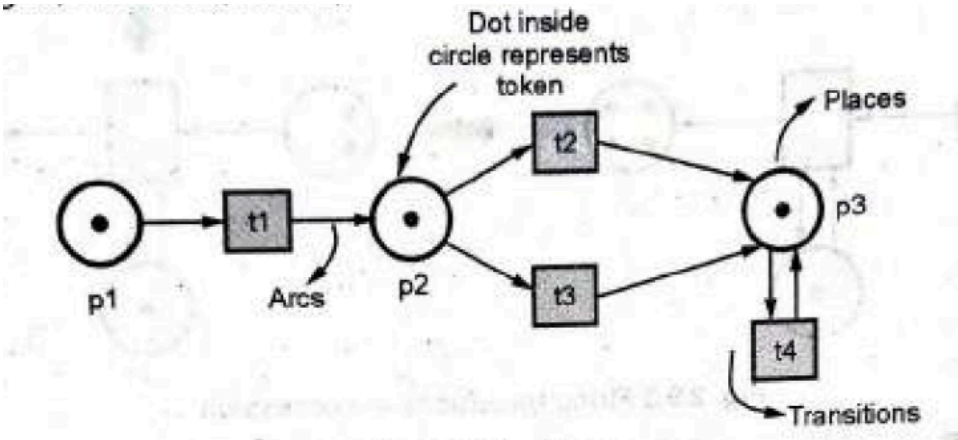
It evaluates a project's or system's practicality. It involves an objective and rational analysis of a potential business or venture to determine its strengths and weaknesses, potential opportunities

and threats, resources required to carry it out, and ultimate success prospects. When judging feasibility, two criteria should be considered: the required cost and expected value.

As the name implies, a feasibility analysis is used to determine the viability of an idea, such as ensuring a project is legally, technically feasible, and economically justifiable. It tells us whether a project is worth the investment—in some cases, it may not be doable. There can be many reasons for this, including requiring too many resources, which not only prevents those resources from performing other tasks but also may cost more than an organization would earn back by taking on a project that isn't profitable.

6. What is the purpose of petrinet? State two advantages of using petri nets.

Ans: Petri nets are a graphical and mathematical modeling tool used to describe and analyze systems that exhibit concurrency, synchronization, and resource sharing. They are particularly useful for modeling distributed and asynchronous systems. Two advantages of using Petri nets are: they can model complex systems with concurrency and synchronization, and they provide a visual and precise representation of system behavior.



Purpose of Petri Nets:

Modeling Concurrent Systems: Petri nets are designed to represent systems where multiple activities can occur simultaneously or independently.

Analyzing System Behavior: They allow for the analysis of system properties like reachability, liveness, and boundedness, helping to identify potential issues like deadlocks or resource conflicts.

Simulating System Dynamics: Petri nets can be used to simulate the dynamic behavior of a system, allowing for the evaluation of different scenarios and the optimization of system performance.

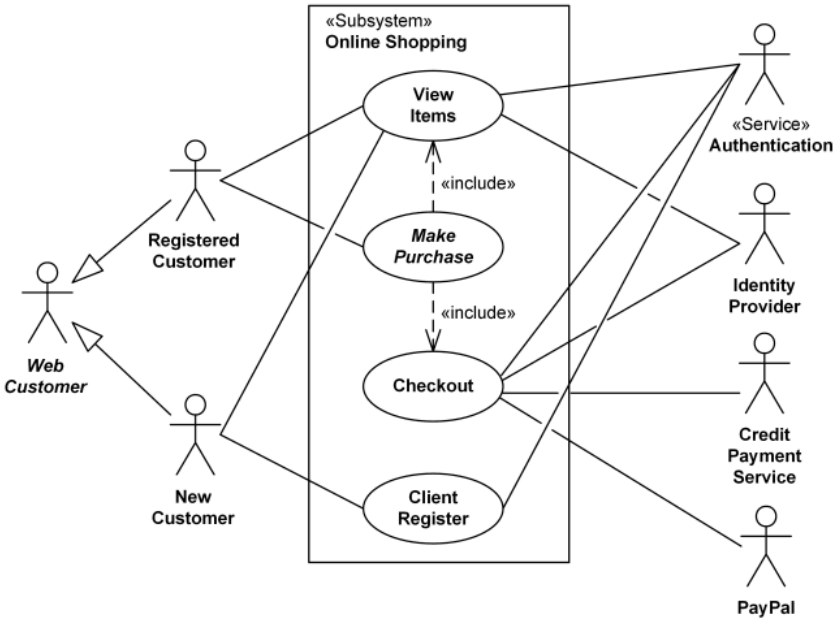
Advantages of using Petri Nets:

1. Ability to Model Concurrency and Synchronization:
Petri nets are well-suited for representing systems where multiple processes can execute in parallel and interact with each other.
2. Graphical and Formal Representation:
They provide a visual and precise way to represent system behavior, making it easier to understand and communicate complex system designs.
3. Formal Analysis:
Petri nets allow for formal analysis of system properties, enabling the identification of potential issues like deadlocks or resource conflicts before implementation.
4. Flexibility and Scalability:
Petri nets can be extended and adapted to model a wide range of systems, from simple to highly complex.

7. Draw a use case diagram for an online shopping which provide provisions for registering, authenticating the customers and also for online payment through any payment gateway like

paypal.

Ans: Explain the following diagram.



8. How does data flow diagram help in design of software system?

Ans: Data Flow Diagram can be represented in several ways. The Data Flow Diagram (DFD) belongs to structured-analysis 1 tools. Data Flow diagrams are very popular because they help us to visualize the major steps and data involved in **software-system processes**.

- is a graphical representation of data flow in any system. It is capable of illustrating incoming data flow, outgoing data flow and store data.
- The DFD depicts both incoming and outgoing data flows and provides a high-level overview of system functionality.
- It is a relatively simple technique to learn and use, making it accessible for both technical and non-technical stakeholders.
- they help us to visualize the major steps involved in software-system processes.

DFDs are categorized into various levels, with each level providing different degrees of detail. The levels are numbered from **0** and onward. The higher the level, the more detailed the diagram becomes. The following are the four levels of DFDs:

0-Level Data Flow Diagram (DFD)

Level 0 DFD is the highest-level diagram, representing the system as a single process with its interactions with external entities. It shows the major processes, data flows, and data stores in the system, without providing any details about the internal workings of these processes. It is also known as the Context Diagram, which abstracts the system's operations and shows how data enters and leaves the system.

Level 1 DFD provides a more detailed view of the system by breaking down the major processes identified in the level 0 Data Flow Diagram (DFD) into sub-processes. Each sub-process is depicted as a separate process on the level 1 Data Flow Diagram (DFD). The data flows and data stores associated with each sub-process are also shown.

Level 1 DFD provides a more detailed view of the system, focusing on key functional aspects. The Context Diagram from Level 0 is expanded into multiple bubbles/processes.

Level 2 DFD further breaks down the sub-processes from Level 1 DFD into additional sub-processes, providing an even more detailed view. This level is useful when dealing with specific requirements or parts of the system that need a closer examination of their processes and interactions.

3-Level is the most detailed level of Data Flow Diagram (DFDs), which provides a detailed view of the processes, data flows, and data stores in the system. This level is typically used for complex systems, where a high level of detail is required to understand the system. It includes detailed descriptions of each process, data flow, and data store, and is usually used when there is a need for a comprehensive understanding of the system.

9. Define UML. What are all the various types of diagram that can be drawn in UML?

Ans: UML, or Unified Modeling Language, is a standardized, general-purpose modeling language used in software engineering to visualize, specify, construct, and document the artifacts of a system. It provides a visual way to represent the design of a system, including its structure, behavior, and interactions. There are different types of UML diagrams, broadly categorized into structural and behavioral diagrams.

Structural Diagrams: These diagrams depict the static structure of a system, showing the classes, interfaces, components, and their relationships. Examples include:

Class Diagram: Shows classes, their attributes, operations, and relationships between them.

Object Diagram: Represents instances of classes and their relationships at a specific point in time.

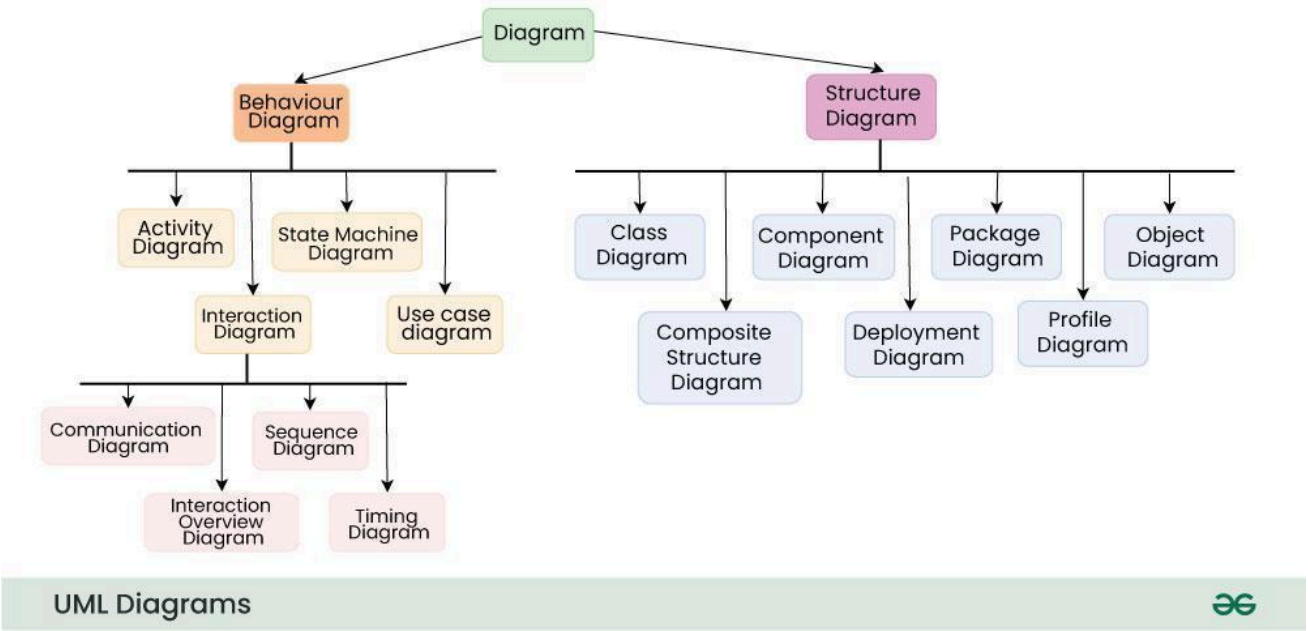
Component Diagram: Illustrates the organization and dependencies between software components.

Deployment Diagram: Shows the physical deployment of software components on hardware nodes.

Package Diagram: Organizes model elements into packages to improve understanding and manage complexity.

Composite Structure Diagram: Shows the internal structure of a class, including its parts and ports.

Profile Diagram: Allows extension of the UML metamodel to adapt to specific domains and platforms.



Behavioral Diagrams: These diagrams focus on the dynamic aspects of a system, showing how objects interact and how the system behaves over time. Examples include:

Use Case Diagram: Represents the interactions between actors and the system, showing the functionality offered by the system.

Activity Diagram: Illustrates the flow of activities and the sequence of actions within a system or use case.

State Machine Diagram: Shows the states of an object and the transitions between those states.

Sequence Diagram: Depicts the sequence of interactions between objects over time.

Communication Diagram: Shows the interactions between objects, emphasizing the messages passed between them.

Interaction Overview Diagram: Provides a high-level view of the control flow of interactions.

Timing Diagram: Focuses on the timing constraints and behavior of objects over a specific period.

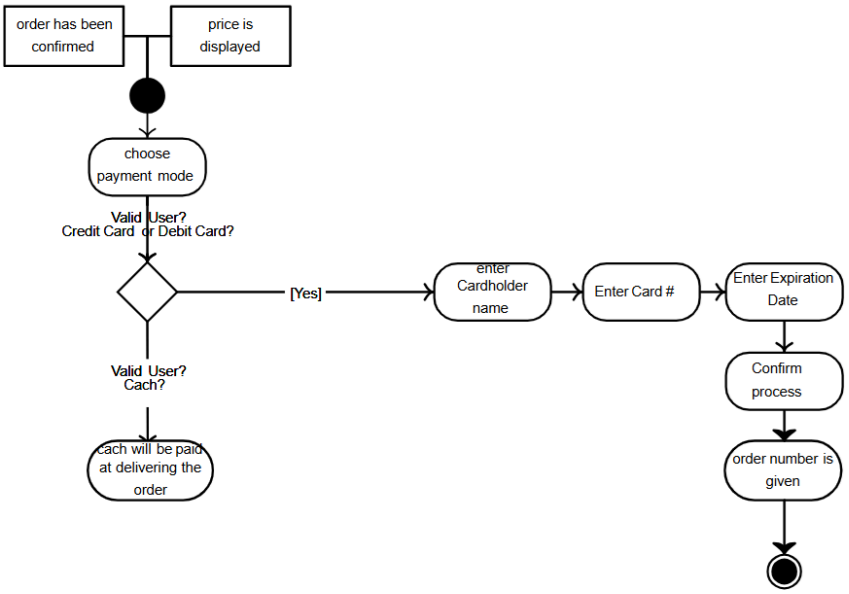
10. When to use use case diagram?

Ans: Use case diagrams are useful in several situations. Here's when you should consider using them:

- When you need to gather and clarify user requirements, use case diagrams help visualize how different users interact with the system.
- If you're working with diverse groups, including non-technical stakeholders, these diagrams provide a clear and simple way to convey system functionality.
- During the system design phase, use case diagrams help outline user interactions and plan features, ensuring that the design aligns with user needs.
- When defining what is included in the system versus what is external, use case diagrams help clarify these boundaries.

11. Define activity diagram?

Ans; An activity diagram, part of the Unified Modeling Language (UML), is a visual representation of a system's behavior by illustrating the flow of activities within a process. It's essentially an advanced flowchart that shows the sequence of actions, including parallel and alternative flows, and can also depict data flow between activities.



Part – B

1. Explain in detail about the functional and non functional requirements. Explain the software requirement engineering process with neat diagram.

Ans: Refer Above question No.4

2. What is requirements elicitation? Briefly describe the various activities performed in requirement elicitation phase with an example of a watch system that facilitates to set time and alarm.

Ans: Requirements elicitation is the process of gathering, analyzing, and defining the needs and expectations of stakeholders for a project or system. It involves understanding what

users, customers, and other stakeholders want and need from a product or system, and then translating those needs into clear, actionable requirements. This process is crucial for ensuring that the final product or system meets the intended goals and objectives.

Gathering:

This involves actively collecting information about the requirements through various techniques like interviews, surveys, workshops, and document analysis.

Analyzing:

Once gathered, the requirements need to be analyzed to identify inconsistencies, conflicts, and dependencies.

Defining:

The analyzed requirements are then translated into a clear and concise format, often using a requirements document or model.

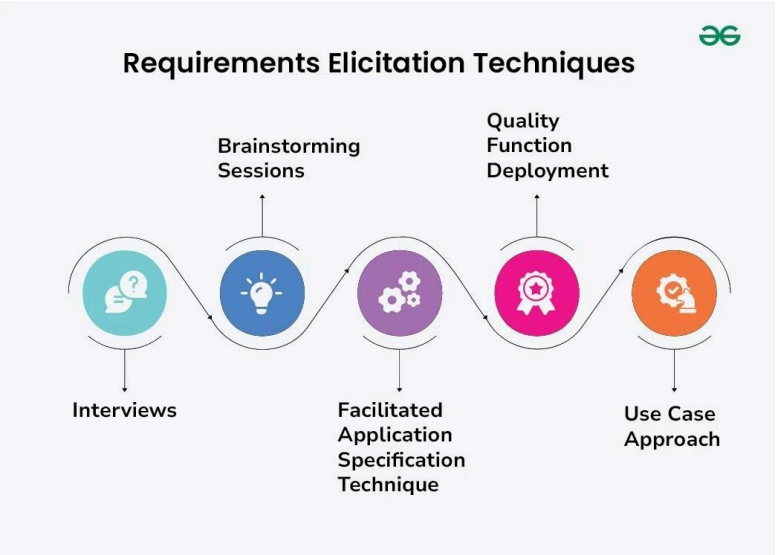
Stakeholders:

The key participants in requirements elicitation include end-users, business owners, project managers, developers, and subject matter experts.

Importance:

Effective requirements elicitation is vital for project success. It helps to:

- Ensure that the developed product or system meets the needs of its users.
- Reduce the risk of project failure due to misunderstandings about requirements.
- Avoid costly rework and delays during development.



3. What are the components of the standard structure for the software requirement document? Explain in detail.

Ans: Software Requirement Specification (SRS) Format as the name suggests, is a complete specification and description of requirements of the software that need to be fulfilled for the successful development of the software system. These requirements can be functional as well as non-functional depending upon the type of requirement. The interaction between different

customers and contractors is done because it is necessary to fully understand the needs of

Document Title

Author(s)
Affiliation
Address
Date
Document Version

customers. Depending upon information gathered after interaction, SRS is developed which describes requirements of software that may include changes and modifications that is needed to be done to increase quality of product and to satisfy customer's demand.

Purpose of this Document - At first, main aim of why this document is necessary and what's purpose of document is explained and described.

Scope of this document - In this, overall working and main objective of document and what value it will provide to customer is described and explained. It also includes a description of development cost and time required.

Overview - In this, description of product is explained. It's simply summary or overall review of product.

General description: In this, general functions of product which includes objective of user, a user characteristic, features, benefits, about why its importance is mentioned. It also describes features of user community.

Functional Requirements : In this, possible outcome of software system which includes effects due to operation of program is fully explained. All functional requirements which may include calculations, data processing, etc. are placed in a ranked order. Functional requirements specify the expected behavior of the system-which outputs should be produced from the given inputs. They describe the relationship between the input and output of the system.

Interface Requirements : In this, software interfaces which mean how software program communicates with each other or users either in form of any language, code, or message are fully described and explained. Examples can be shared memory, data streams, etc.

Performance Requirements : In this, how a software system performs desired functions under specific condition is explained. It also explains required time, required memory, maximum error rate, etc. The performance requirements part of an SRS specifies the performance constraints on the software system. All the requirements relating to the performance characteristics of the system must be clearly specified.

Design Constraints : In this, constraints which simply means limitation or restriction are specified

and explained for design team. Examples may include use of a particular algorithm, hardware and software limitations, etc.

Non-Functional Attributes : In this, non-functional attributes are explained that are required by software system for better performance. An example may include Security, Portability, Reliability, Reusability, Application compatibility, Data integrity, Scalability capacity, etc.

Preliminary Schedule and Budget : In this, initial version and budget of project plan are explained which include overall time duration required and overall cost required for development of project.

Appendices: In this, additional information like references from where information is gathered, definitions of some specific terms, acronyms, abbreviations, etc. are given and explained.

4. Explain Petri Net in details. Draw a Petri Net that depicts the operation of an "Automated Teller Machine". State the functional requirements you are considering.

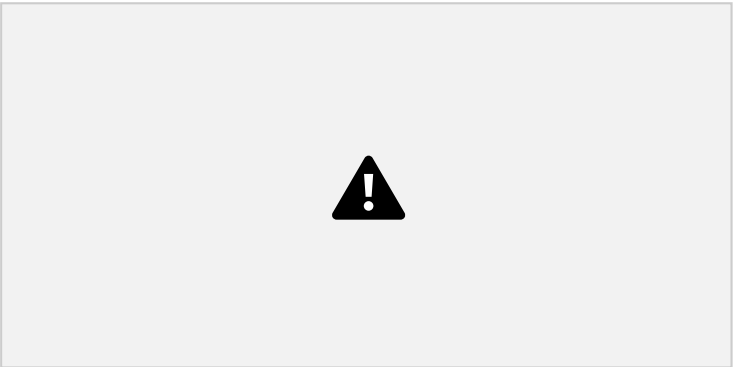
   Ans: Refer Above question No.6.

5. Explain with an example, how use case modeling is used to describe functional requirements. Identify the actors, scenarios and use cases for example.

   Ans: Refer Above question No.4

A Use Case diagram for Online Shopping System:

- **Actors:**
  - Customer
  - Admin
- **Use Cases:**
  - Browse Products
  - Add to Cart
  - Checkout
  - Manage Inventory (Admin)
- **Relations:**
  - The Customer can browse products, add to the cart, and complete the checkout.
  - The Admin can manage the inventory



6. Describe the strategies used to identify the conceptual classes.

Ans: To identify conceptual classes, strategies include reusing existing models, utilizing category lists, and analyzing noun phrases within textual descriptions. These approaches help to represent the key concepts and relationships within a specific domain, forming the foundation for object-oriented modeling.

Strategies for Identifying Conceptual Classes:

1. Reusing or Modifying Existing Models:

This is often the easiest approach, as pre-built models exist for common domains like inventory,

finance, and healthcare. Leveraging these models can save time and effort, but may need to be adapted to specific project requirements.

2. Using a Category List:

Creating a list of candidate conceptual classes based on domain knowledge is a useful technique. This list can be refined and prioritized during the analysis phase, ensuring that all relevant concepts are considered.

3. Identifying Noun Phrases:

Analyzing textual descriptions of the domain, such as user stories or requirements documents, can reveal potential classes. Nouns and noun phrases often represent entities or concepts within the system, making them good candidates for classes.

4. Analyzing Nouns and Noun Phrases in Requirements:

This strategy involves identifying nouns and noun phrases in problem statements or requirements documents to pinpoint potential classes. For example, nouns referring to persons, places, or entities can be mapped to objects, and singular proper nouns can also be identified as potential classes.

5. Analyzing Use Cases:

Examining use cases, particularly the main success scenario (or basic flow), can reveal the key actions and entities involved, which can then be translated into conceptual classes and their relationships.

7. Describe the steps to create a domain model used for representing the conceptual classes.

Ans: A domain model is a representation of real-world conceptual classes - not of software components. A domain model is the most important artifact created during object-oriented analysis.

Domain model is a structured visual representation of interconnected concepts or real-world objects that incorporates vocabulary, key concepts, behavior, and relationships of all of its entities.

The key word in the above definition for me personally is *visual*. My brain performs at a maximum if a theoretical concept can be visualized in a digestible and somewhat eye-pleasing manner. Most of the time a domain model is illustrated with a set of class diagrams which may show:

1. domain objects or conceptual classes
2. associations between conceptual classes
3. attributes of conceptual classes

Building a Domain Model

Let's take a look at an example of modeling a particular domain and analyze its components. We will be dealing with a simplified Vehicle domain. Here's a verbal description that will serve as a base for our model:

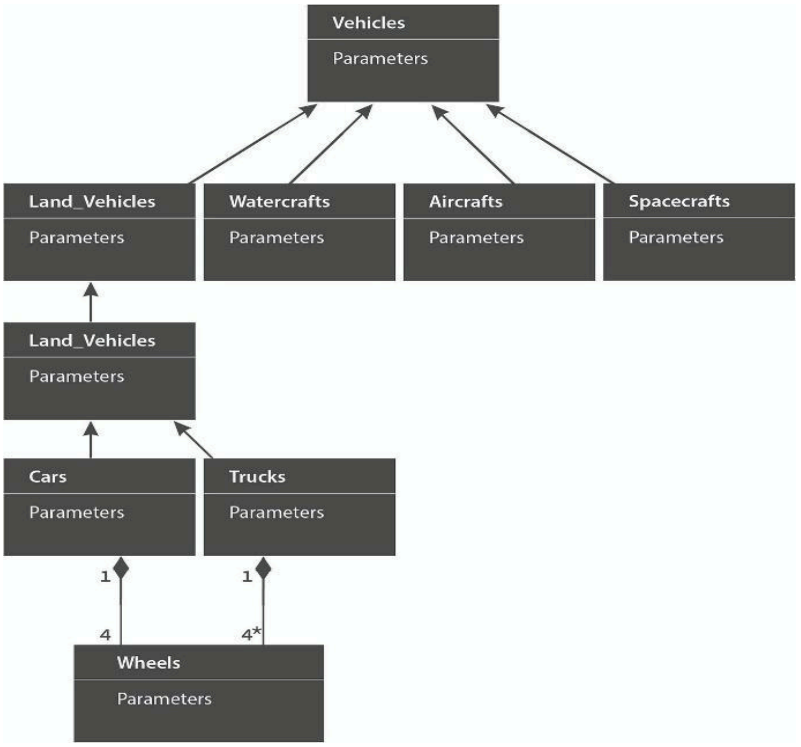
A vehicle is a mobile machine that transports people or cargo. Typical vehicles include wagons, bicycles, motor vehicles (motorcycles, cars, trucks, buses), railed vehicles (trains, trams), watercraft (ships, boats), aircraft and spacecraft. Land vehicles are classified broadly by what is used to apply steering and drive forces against the ground: wheeled, tracked, railed or skied.

Using Noun Phrase Identification to Single Out Conceptual Classes

We will be applying a useful technique — linguistic analysis: identifying the nouns and noun phrases in textual description of a domain, and considering them as candidate conceptual classes or attributes.

Machine ,wagons, bicycles, motor vehicles, motorcycles, cars, trucks, buses, railed vehicles, trains, trams, watercraft, ships, boats, aircraft, spacecraft. Land vehicles, wheeled, tracked, railed, skied.

Transforming the above list into classes with the corresponding Class_names, this is what our Vehicle domain model may look like:

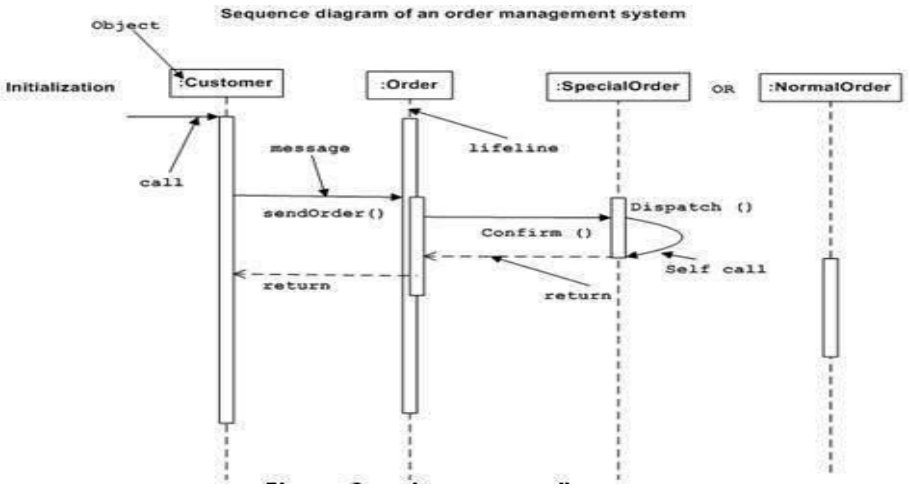


8. What are system sequence diagram? What is the relationship between SSDs and Use cases? Explain with an example.

Ans: A sequence diagram simply depicts interaction between objects in a sequential order i.e. the order in which these interactions take place.

- Sequence diagrams describe how and in what order the objects in a system function.

Sequence diagrams and use cases are related in that sequence diagrams are often used to model the detailed interactions within a use case	A use case describes what a system does from the user's perspective, while a sequence diagram shows the step-by-step interactions between objects (including users and the system) to realize that use case.
Sequence diagrams are a type of UML interaction diagram that visually represents the flow of interactions between objects over time. They show the sequence of messages exchanged between objects to fulfill a particular use case or scenario.	Use cases are high-level descriptions of how users interact with a system to achieve specific goals. They focus on the functionality of the system and the actors involved.
A sequence diagram can be created for each use case to show the specific steps and interactions required to complete that use case. The sequence diagram refines the use case by providing a detailed view of the object interactions	A use case diagram, which shows the relationships between use cases and actors, can be used to identify which use cases need to be further detailed using sequence diagrams. Each use case in a use case diagram can potentially have one or more corresponding sequence diagrams

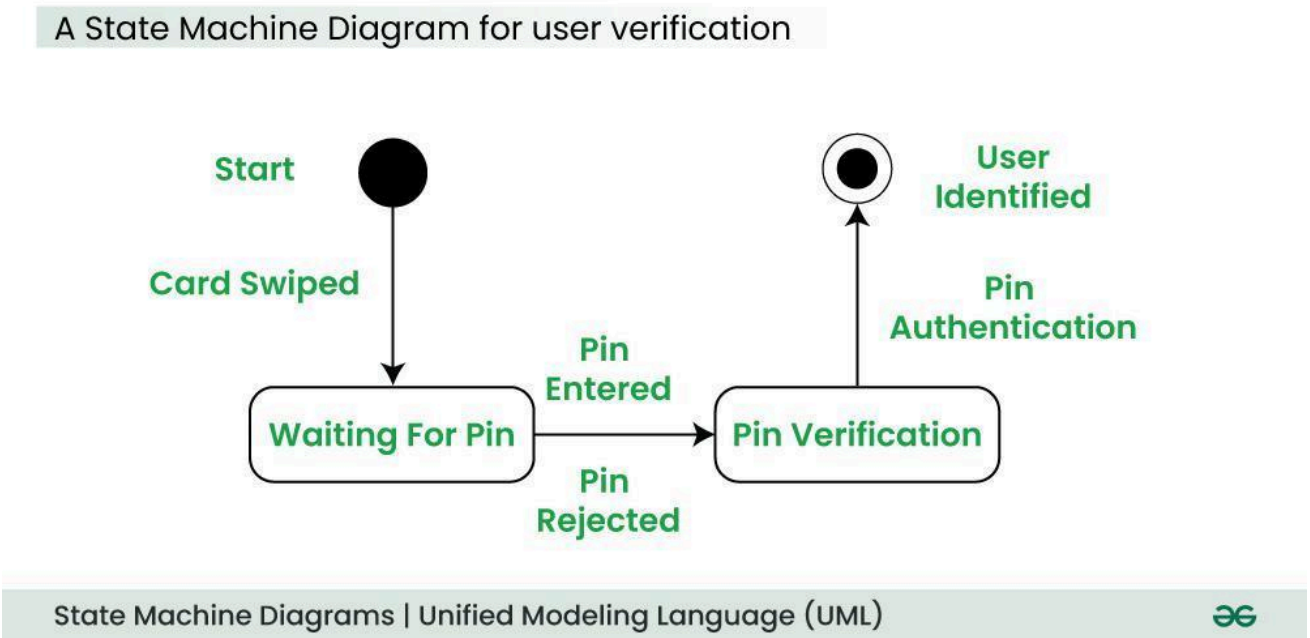


9. Explain the state chart diagram with a suitable example. Also define its components and use.

Ans:

A State diagram is a UML diagram which is used to represent the condition of the system or part of the system at finite instances of time. It's a behavioral diagram and it represents the behavior using finite state transitions.

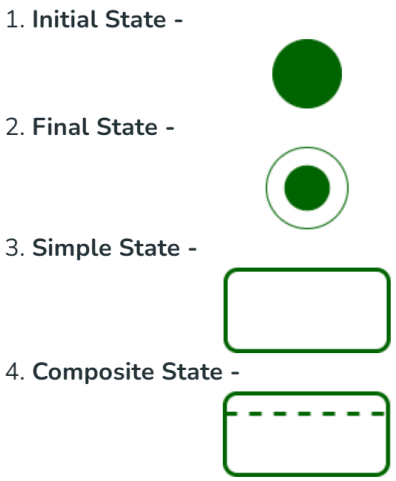
- State Machine diagrams are also known as State Diagrams and State-Chart Diagrams. These both terms can be used interchangeably.
- A state machine diagram is used to model the dynamic behaviour of a class in response to time and changing external stimuli(events that cause the system to change its state from one to another).
- We can say that every class has a state but we don't model every class using State Machine diagrams.

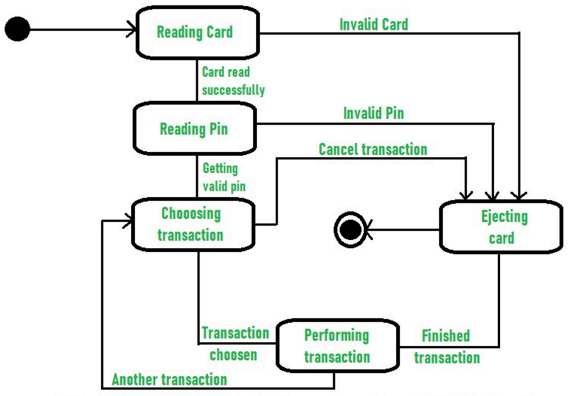


The State Machine Diagram above shows the different states in which the verification sub-system or class exists for a particular system.

Key Elements in a State Chart Diagram

1. Transitions: Represented as arrows, indicating the change between states.
2. Initial State: The starting point of the system, usually represented by a filled circle with an arrow pointing to the initial state.
3. Final State: system ends its operation, represented by a concentric circle with an arrow pointing to it.
4. Events: Trigger transitions between states
5. Actions: Tasks that occur when a state is entered or exited





State Transition Diagram for ATM System

Part – C

1. Develop an online railway reservation system, which allows the user to select route, book/cancel tickets using net banking/ credit /debit cards. The site also maintains the history of the passengers. For the above system, list and draw the use case scenario and model the above specification.
2. Develop the software requirement document for the following requirement. A coffee vending machine serves coffee to customers. A customer can choose a type of coffee among a list often options, supply the amount required and get served. Each coffee is prepared by adding units of hot water, coffee powder, milk and sugar. The recipe for each coffee is stored. Develop a usecase diagram, Activity diagram and Sequence diagram.
3. Model a class diagram for "Banking System" State the functional requirements you are considering.
4. Draw Sequence and collaboration diagram for online course reservation system.
5. Develop the process of ordering a pizza over the phone. Draw the use case diagram and also sketch the activity diagram representing each step of the process, from the moment you pick up the phone to the point where you start eating the pizza. Include activities that others need to perform. Add exception handling to the activity diagram you developed. Consider at least two exceptions (e.g. delivery person wrote down wrong address, deliver person brings wrong pizza).